Universidade de São Paulo (USP)
Instituto de Ciências Matemáticas e Computação (ICMC)

Trabalho 1 - Simulação paralela da propagação direcional de incêndio com zonas de contenção

Computação de Alto Desempenho (SSC0903)

Hector Yolvi Herrera Cruz	NUSP: 17819731
Luan Benson Ribeiro	NUSP: 13672085
Nicolas Amaral dos Santos	NUSP: 16304033
Pedro Antonio Bruno Grando	NUSP: 12547166
Pedro Bizon Dania	NUSP: 11812667

24 de setembro de 2026

São Carlos - SP

Sumário

1 Introdução e Descrição do Modelo **2**
1.1 Representação do Domínio Espacial 2
1.2 Dinâmica de Propagação e Potencial de Ignição 2

2 Arquitetura da Solução Sequencial **3**

3 Arquitetura da Solução Paralela **3**
3.1 Organização de Memória e Localidade de Cache 3
3.2 Região Paralela Persistente 4
3.3 Otimizações do Núcleo Computacional 4
3.3.1 Pré-computação dos Efeitos do Vento 4
3.3.2 Tratamento Separado das Bordas 4
3.3.3 Acessos Locais às Linhas do Grid 4
3.3.4 Vetorização do Loop Interno 5
3.4 Sincronização, Ativação de Zonas e Double-Buffering 5
3.4.1 Zonas de Contenção e Reductions 5
3.4.2 Double-Buffering e Atualização do Estado 5

4 Resultados Experimentais e Análise de Desempenho **6**
4.1 Ambiente Experimental 6
4.2 Validação Numérica e Checksum 6
4.3 Análise de Escalabilidade Forte (Speedup e Eficiência) 6
4.4 Comparação das Políticas de Escalamento (*Schedules*) 7

5 Conclusão **8**

1 Introdução e Descrição do Modelo

A simulação computacional de incêndios florestais é um problema de física computacional de grande relevância, exigindo alto rendimento de processamento devido ao elevado número de células e passos temporais necessários para simular grandes extensões territoriais. O modelo estudado é baseado em um autômato bidimensional determinístico, estruturado sobre uma matriz de dimensões $L \times C$, onde L representa o número de linhas e C o número de colunas.

1.1 Representação do Domínio Espacial

Cada célula da matriz (i, j), onde $0 \leq i < L$ e $0 \leq j < C$, possui atributos fundamentais que ditam a dinâmica de propagação:

1. **Tipo de Cobertura (*cov*):** Define a inflamabilidade e o tempo de queima da vegetação. É dividida em Água ($cov = 0$), Solo Exposto ($cov = 1$), Vegetação Rasteira ($cov = 2$) e Floresta ($cov = 3$).

2. **Umidade (*umid*):** Percentual inteiro $U \in [0, 100]$ que reduz o potencial de ignição recebido pela célula.

3. **Estado (*est*):** Define a situação corrente da célula no tempo p, assumindo valores discretos: Não Combustível (0), Intacta (1), Em Chamas (2), Queimada (3) ou Contenção (4).

4. **Tempo Restante de Queima (*tempo*):** Contador decrescente ativado quando a célula entra em chamas. Células do tipo Vegetação Rasteira queimam durante 2 passos, enquanto células de Floresta queimam por 4 passos.

1.2 Dinâmica de Propagação e Potencial de Ignição

A propagação do fogo ocorre através da vizinhança de Moore (8 células adjacentes). Células fora dos limites da matriz são desconsideradas. O potencial de ignição I incidente sobre uma célula intacta (i, j) no passo p é calculado somando-se as contribuições de todas as suas células vizinhas $v \in \text{Moore}(i, j)$ que estejam no estado "Em Chamas"($est_v = 2$).

Para cada vizinho em chamas, calcula-se o vetor relativo de propagação $\vec{P} = (\text{prop_linha}, \text{prop_coluna}) = (i - i_v, j - j_v)$. O peso básico $P_{\text{básico}}$ assume valor 10 para conexões ortogonais e 7 para posições diagonais. A influência direcional do vento $\vec{V} = (\text{vento_linha}, \text{vento_coluna})$ com intensidade $V \in [0, 5]$ é computada através do produto escalar de alinhamento A:

$$A = \text{prop_linha} \times \text{vento_linha} + \text{prop_coluna} \times \text{vento_coluna} \quad \in \{-2, -1, 0, 1, 2\} \tag{1}$$

O peso final do vizinho P_v e o acúmulo total de potencial S são obtidos por:

$$P_v = \max(1, P_{\text{básico}} + V \times A) \tag{2}$$

$$S = \sum_{v \in \text{Moore}(i,j), est_v = 2} P_v \tag{3}$$

A ignição da célula (i, j) ocorre se, e somente se, o potencial truncado I atingir ou superar o limiar global $LIMIAR$:

$$I = \left\lfloor \frac{S \times F_{\text{combustível}} \times (100 - U)}{100} \right\rfloor \geq LIMIAR \tag{4}$$

onde $F_{\text{combustível}} = 8$ para Vegetação Rasteira e $F_{\text{combustível}} = 12$ para Floresta.

2 Arquitetura da Solução Sequencial

A versão sequencial do projeto foi feita com foco em garantir que a simulação fosse totalmente determinística e correta. Ela serviu como base de validação para a implementação paralela logo em seguida.

Para representar o mapa da floresta, decidimos guardar a malha bidimensional em vetores unidimensionais na memória. Para acessar uma célula específica na linha i e coluna j, usamos a fórmula direta de índice: $indice = i \times C + j$. Fazer isso ajuda a evitar os atrasos e saltos de memória que acontecem quando usamos matrizes dinâmicas tridimensionais, além de aproveitar muito melhor a memória *cache* do processador.

A inicialização segue a ordem exata exigida nas especificações do trabalho: varremos a matriz linearmente (linha por linha) e chamamos a função `rand_r(&seed)` para definir primeiro a cobertura do solo e, no passo imediatamente seguinte, a umidade daquela mesma célula.

Para a passagem do tempo, usamos a técnica de *Double-Buffering*. O código isola dois espaços na memória: o `estado_atual` e o `proximo_estado` (além dos seus respectivos tempos de queima). A mecânica de cada turno funciona assim:

1. **Ativação de Zonas:** Primeiro, checamos o vetor de ativação para colocar em estado de contenção as células marcadas para aquele passo.

2. **Propagação:** Olhamos para os 8 vizinhos (vizinhança de Moore) lendo apenas as informações do estado atual para calcular o potencial direcional do fogo.

3. **Gravação:** Qualquer nova ignição ou redução no tempo de queima é gravada apenas no `proximo_estado`. Isso garante que a mudança em uma célula não afete o cálculo da sua vizinha durante o mesmo laço temporal.

4. **Atualização e Parada:** No fim do passo temporal, nós apenas invertemos os ponteiros dos *buffers* (um *swap* rápido na memória) e testamos se o incêndio acabou ou se chegamos no limite de P passos.

3 Arquitetura da Solução Paralela

A solução escolhida foi organizada de forma a explorar o paralelismo inerente à estrutura da simulação. A cada passo de tempo, o estado de uma célula é calculado a partir das informações disponíveis no passo anterior. As células de um mesmo passo podem ser processadas de forma paralela enquanto os passos temporais são executados sequencialmente. É justamente a estrutura facilmente divísivel espacialmente que é explorada para dividir a carga entre threads diferentes.

3.1 Organização de Memória e Localidade de Cache

A matriz do terreno é armazenada em vetores unidimensionais contíguos, em vez de utilizar uma estrutura bidimensional, baseada em dois níveis de ponteiros. Com isso, o acesso a uma célula é obtido a partir da conversão das coordenadas de linha e coluna em um único índice. Essa organização simplifica o acesso à memória e favorece a localidade espacial, já que células vizinhas ocupam posições próximas.

No âmbito das estruturas de dados, estes são separados de acordo com sua função. Informações que permanecem imutáveis ao longo da execução, como a cobertura do terreno e a umidade, são separadas de dados que são constantemente atualizados, como o estado atual do

incêndio. Essa separação facilita o processamento da estrutura, ao reduzir acessos à memória, além de permitir utilizar tipos de dados menores, adaptados à necessidade de cada operação. Isso, no geral, reduz a quantidade de memória movimentada durante a simulação.

3.2 Região Paralela Persistente

Conforme as instruções do trabalho, o código possui uma única região OpenMP que permanece ativa durante toda a execução da propagação. Com isso, threads são criadas uma única vez e vivem toda a duração da simulação, evitando a repetição do overhead associado à criação e finalização delas.

Dentro dessa região, cada passo é dividido em etapas. As partes com vários cálculos idênticos, e que podem ser executadas independentemente, são distribuídas entre as threads. As operações que precisam ocorrer apenas uma vez por passo, por outro lado, são realizadas sequencialmente por uma thread. Esse modelo facilita a sincronização entre as diferentes etapas da simulação, especialmente tendo em conta que as operações únicas normalmente são executadas em preparação ou finalização dos cálculos paralelos.

"default(none)"é utilizado, o que está conforme às requisições do trabalho. Essa diretiva reforça essa organização ao exigir que o compartilhamento das variáveis seja definido explicitamente, não permitindo que o compilador o assuma implicitamente. Além de tornar a estrutura do programa mais clara, isso ajuda a identificar possíveis problemas de escopo em regiões paralelas.

3.3 Otimizações do Núcleo Computacional

Como a atualização da grade é realizada repetidamente para todas as células, pequenos atrasos ou computações ineficientes nesse trecho se acumulam ao longo da execução. Como resultado, grandes penalidades de tempo são introduzidas na execução. Portanto, as otimizações adotadas nessa zona procuram reduzir trabalho repetitivo e tornar os acessos à memória otimizáveis para o compilador e para o processador.

3.3.1 Pré-computação dos Efeitos do Vento

Os parâmetros associados ao vento não mudam durante a simulação. Por isso, seus efeitos sobre a contribuição dos vizinhos podem ser calculados previamente e reutilizados durante toda a simulação. Com isso, o núcleo se concentra no processamento efetivo das células, sem repetir cálculos já feitos.

3.3.2 Tratamento Separado das Bordas

As células localizadas nas bordas da grade são um caso particular, pois nem todos os seus vizinhos existem. Para evitar que essa verificação seja necessária para todas as células, o código trata as bordas separadamente. Assim, o processamento principal no interior da matriz pode ser otimizado.

Essa organização reduz a quantidade de condições dentro do laço mais denso da simulação. O princípio é similar a alguns algoritmos de stencil, nos quais os casos de fronteira são tratados de forma específica. Isso permite manter o caminho principal mais simples, logo mais facilmente otimizável pelo compilador.

3.3.3 Acessos Locais às Linhas do Grid

No processamento do interior da matriz, os dados da linha atual e de suas linhas vizinhas são acessados por meio de referências locais. Em vez de reconstruir a cada vez o endereço

completo de cada elemento, a posição inicial de cada linha é determinada uma vez e os elementos seguintes são acessados de forma sequencial.

Essa abordagem é compatível com a organização contígua da memória e favorece tanto a localidade de cache quanto a otimização dos acessos realizada pelo compilador. O uso de "restrict"em conjuntos de dados independentes também fornece ao compilador informações adicionais sobre possíveis aliasing. Isso permite otimizações mais agressivas sem alterar o comportamento da simulação.

3.3.4 Vetorização do Loop Interno

O processamento das células interiores também foi pensado de forma a favorecer a vetorização. As operações realizadas em uma célula dependem dos valores do passo anterior e produzem resultados em posições distintas da matrix seguinte. Portanto, várias células podem ser tratadas simultaneamente por instruções vetoriais.

A diretiva "omp simd"indica essa possibilidade ao compilador. O cálculo da transição foi mantido em uma forma simples, reduzindo a necessidade de condicionais e branchs dentro do laço principal. O objetivo, nesse caso, não era alterar a lógica do algoritmo, mas organizar sua execução de modo que várias células possam ser processadas em paralelo dentro de cada thread.

3.4 Sincronização, Ativação de Zonas e Double-Buffering

A sincronização da simulação é pensada de forma que durante cada passo, todas as threads trabalham sobre o mesmo estado de referência e produzem conjuntamente o próximo estado. Isso evita que o resultado de uma célula seja utilizado precipitadamente por outra durante a mesma iteração.

3.4.1 Zonas de Contenção e Reductions

As zonas de contenção são foram escolhidas de forma a evitar que threads passem muito tempo esperando umas pelas outras. O trabalho associado a uma zona é distribuído entre as threads, permitindo que diferentes partes de sua área sejam processadas simultaneamente. Usando reductions em contadores relevantes, uma variável local registra as células efetivamente afetadas e os resultados são posteriormente agregados de forma segura.

Essa estratégia evita a necessidade de tratar a ativação das zonas como uma operação sequencial e mantém a mesma divisão de trabalho utilizada nas demais etapas da simulação.

3.4.2 Double-Buffering e Atualização do Estado

Para calcular o próximo estado do grid, a implementação mantém dois conjuntos de dados. O primeiro representa o estado atual e o segundo recebe os resultados do passo seguinte. Ao final da iteração, essas variáveis são trocadas.

Esse mecanismo elimina dependências de leitura e escrita dentro de um mesmo passo, sendo particularmente adequado para algoritmos de stencil. Como cada thread escreve em células diferentes do próximo estado, não é necessário proteger essas escritas individualmente com regiões críticas ou operações atômicas.

As estatísticas do passo também são acumuladas de forma distribuída e combinadas ao final da etapa, com o uso de reductions. Depois dessa atualização, a troca dos buffers e a verificação da condição de parada são realizadas. A sincronização implícita entre os passos garante que todas as threads tenham terminado de produzir o próximo estado antes que ele passe a ser utilizado como entrada.

4 Resultados Experimentais e Análise de Desempenho

4.1 Ambiente Experimental

4.2 Validação Numérica e Checksum

A validação entre as versões sequencial (`fire_seq`) e paralela (`fire_omp`) foi realizada utilizando arquivos de entrada padronizados (entrada_carga_pequena). A Tabela 1 comprova que a versão paralela produz resultados rigorosamente idênticos à versão sequencial, garantindo o determinismo do modelo.

Tabela 1: Validação determinística de checksum e estatísticas finais entre versões.

Versão	**Threads**	**Passos**	**Total Ignições**	**Pico Ignições**	**Checksum Final**	**Status**
Sequencial	1	80	30171	79 682	17073017104646724864	Referência
Paralela	1	80	30171	79 682	17073017104646724864	Idêntico
Paralela	2	80	30171	79 682	17073017104646724864	Idêntico
Paralela	4	80	30171	79 682	17073017104646724864	Idêntico
Paralela	8	80	30171	79 682	17073017104646724864	Idêntico
Paralela	16	80	30171	79 682	17073017104646724864	Idêntico

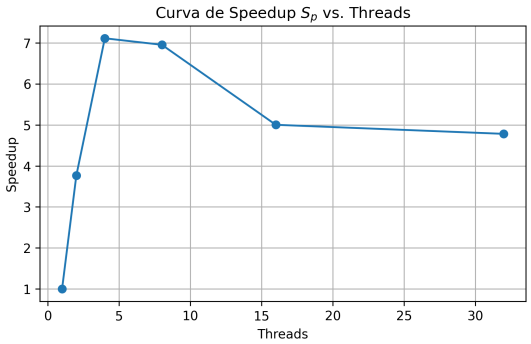
4.3 Análise de Escalabilidade Forte (Speedup e Eficiência)

A escalabilidade forte avalia o ganho de velocidade ao fixar o tamanho do problema (nesse caso: matriz 400×500) e aumentar o número de threads p. As métricas de **Speedup** (S_p) e **Eficiência** (E_p) são calculadas por:

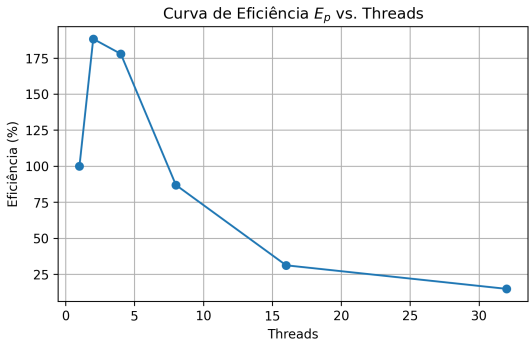
$$S_p = \frac{T_{\text{seq}}}{T_p}, \quad E_p = \frac{S_p}{p} \times 100\% \tag{5}$$

Tabela 2: Tempos de execução do núcleo cronometrado, Speedup e Eficiência.

Threads (p)	**Tempo Médio (s)**	**Desvio Padrão (s)**	**Speedup (S_p)**	**Eficiência (E_p)**
Sequencial	0.191896	0.001901	1.00x	100.0%
2	0.050965	0.000314	3.765x	188.26%
4	0.026970	0.000111	7.115x	177.88%
8	0.027583	0.000300	6.957x	86.96%
16	0.038346	0.000384	5.004x	31.28%
32	0.040119	0.000836	4.783x	14.95%



(a) Curva de Speedup S_p vs. Threads.



(b) Curva de Speedup S_p vs. Threads.

Figura 1: Gráficos de desempenho e escalabilidade da solução paralela em OpenMP.

4.4 Comparação das Políticas de Escalamento (*Schedules*)

A Tabela 3 compara o tempo de execução do núcleo em função das diferentes políticas de balanceamento de carga do OpenMP (`static`, `dynamic` e `guided`).

Tabela 3: Desempenho (tempo em segundos) para diferentes políticas de agendamento OpenMP.

Política de Schedule	**Chunk Size**	**4 Threads**	**8 Threads**	**16 Threads**
`static`	Padrão (L/T)	`0.033201`	`0.028100`	`0.038423`
`static`	16	`0.033128`	`0.027650`	`0.037409`
`dynamic`	16	`0.033099`	`0.033700`	`0.037444`
`guided`	16	`0.033265`	`0.033540`	`0.037601`

5 Conclusão

A simulação mostrou que a estratégia de paralelizar o código em memória compartilhada com OpenMP foi bem-sucedida. Conseguimos comprovar que a versão paralela manteve o determinismo exigido pelo modelo físico: os *checksums* finais, o total de novas ignições e a quantidade de áreas queimadas bateram perfeitamente com a versão sequencial em todos os testes realizados.

Ao analisarmos o desempenho, ficou clara a redução drástica no tempo de execução do núcleo da simulação. Contudo, a escalabilidade apresentou um comportamento sub-linear, limitando-se por gargalos físicos e teóricos, destacando-se:

- **Restrições de Memória (*Memory-bound*):** Como a simulação exige constante movimentação e cópia de dados da matriz, a largura de banda da memória limitou os ganhos lineares de velocidade.
- **Lei de Amdahl:** O ganho de *speedup* esbarrou nas partes estritamente sequenciais do código, especialmente a sincronização necessária para trocar os ponteiros no *Double-Buffering*, a soma final das *reductions* e os testes da condição de parada.
- **Fração Serial Efetiva (Karp-Flatt):** Os testes mostraram que escalar demais o número de *threads* acaba gerando um custo extra alto. O tempo gasto com o gerenciamento e com as barreiras de sincronização entre elas começa a prejudicar a eficiência geral da máquina.

A análise de escalabilidade forte permitiu avaliar o comportamento da aplicação ao manter o tamanho do problema fixo e aumentar progressivamente o número de *threads*. Os resultados demonstraram ganhos expressivos de desempenho nas primeiras configurações avaliadas, porém com redução da eficiência para maiores níveis de paralelismo. Esse comportamento evidencia a influência dos custos de sincronização, das limitações de largura de banda de memória e das parcelas sequenciais da aplicação, conforme previsto pela Lei de Amdahl. Dessa forma, embora o aumento de recursos computacionais reduza o tempo de execução, os ganhos deixam de crescer proporcionalmente após determinado ponto.

Além disso, a comparação entre diferentes políticas de escalonamento do OpenMP mostrou que a escolha do *schedule* influencia diretamente o desempenho da aplicação. A política *static* apresentou resultados competitivos devido à característica regular da distribuição de trabalho da simulação, em que as regiões processadas possuem custo computacional semelhante. Já as políticas *dynamic* e *guided*, apesar de oferecerem maior flexibilidade para aplicações com desequilíbrio de carga, apresentaram custos adicionais relacionados ao gerenciamento dinâmico das tarefas. Esses resultados reforçam a importância de selecionar uma estratégia de escalonamento compatível com as características do problema.

Mesmo com essas limitações naturais da arquitetura do *hardware*, o projeto cumpriu todos os requisitos. Manter a região paralela persistente desde o início e fatiar o processamento da matriz provou ser a saída mais eficiente, permitindo processar mapas gigantes em pouco tempo, sem condições de corrida e mantendo a integridade da simulação.